🕵️ Zaawansowany MLlib

Tuning hiperparametrów, CrossValidator i walka z class imbalance

Python w Zastosowaniach Big Data | Wykład 10

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. Recap: Pipeline, Estimator, Transformer (Wykład 9)
2. Czym są hiperparametry i dlaczego zgadywanie ich ręcznie to zły pomysł
3. `ParamGridBuilder` — budowanie siatki hiperparametrów
4. `CrossValidator` — k-fold cross-validation w skali Spark
5. `TrainValidationSplit` — szybsza, ale mniej solidna alternatywa
6. Class imbalance — problem, który psuje Twój model po cichu
7. Ważenie klas (`weightCol`) i undersampling w Spark
8. `GBTClassifier` — Gradient-Boosted Trees w MLlib
9. AUC-ROC vs AUC-PR — którą metrykę wybrać?
10. Zapis i odczyt `PipelineModel`
11. Skalowanie CV: cache, liczba partycji, koszt obliczeniowy
12. Najczęstsze pułapki
13. Podsumowanie i co dalej

> 🎯 **Po tym wykładzie:** Będziesz potrafić automatycznie dostroić model MLlib i poprawnie ocenić go na mocno niezbalansowanych danych!

🔄 Recap: Co umiemy z Wykładu 9?

Koncept	Opis	Status
`Transformer` / `Estimator`	`transform()` vs `fit()`	✅ Umiemy!
`VectorAssembler`	łączenie features w wektor	✅ Umiemy!
`StringIndexer` / `OneHotEncoder`	kodowanie kategorii	✅ Umiemy!
`Pipeline`	łańcuch kroków fit/transform	✅ Umiemy!
`BinaryClassificationEvaluator`	AUC-ROC	✅ Umiemy!

> 💪 Świetna baza! Dziś dodajemy **automatyczny tuning** i uczymy się radzić sobie z **danymi niezbalansowanymi**.

💡 Hook — Dlaczego tu jesteśmy?

Wytrenowałeś/aś model wykrywania fraudów. Ma **99% accuracy**! Szef jest zachwycony... dopóki nie zapyta, ile fraudów faktycznie wykrył 🤨

```
# Model, który zawsze przewiduje "brak fraudu"
def naive_model(transaction):
    return 0  # zawsze "nie fraud"

# Accuracy przy fraud rate 1%?
# 99% transakcji to nie-fraudy -> model ma 99% accuracy!
# ...i wykrywa ZERO fraudów 💀
```

> Dziś nauczymy się, jak **poprawnie** trenować i oceniać modele na danych, gdzie klasa, na której nam zależy, to zaledwie 1-2% wszystkich przykładów.

🤔 Think About It

Pytanie: Pracujesz dla banku. Model wykrywania fraudów ma 99.3% accuracy. Czy to dobry model?

Czym są hiperparametry?

Hiperparametr to ustawienie modelu, którego **nie uczymy się z danych** — sami je wybieramy przed treningiem.

Model	Przykładowe hiperparametry
`RandomForestClassifier`	`numTrees`, `maxDepth`, `minInstancesPerNode`
`GBTClassifier`	`maxIter`, `maxDepth`, `stepSize`
`LogisticRegression`	`regParam`, `elasticNetParam`

> ⚠️ Ręczne zgadywanie hiperparametrów = godziny straconego czasu i modele gorsze niż być mogły!

🔧 `ParamGridBuilder` — buduj siatkę automatycznie

```
from pyspark.ml.tuning import ParamGridBuilder
from pyspark.ml.classification import RandomForestClassifier

rf = RandomForestClassifier(featuresCol="features", labelCol="is_fraud")

grid = (
    ParamGridBuilder()
    .addGrid(rf.numTrees, [50, 100, 200])
    .addGrid(rf.maxDepth, [5, 8, 12])
    .build()
)

print(len(grid))  # 👈 3 x 3 = 9 kombinacji do przetestowania!
```

> 🔑 Każda kombinacja = osobny model do wytrenowania i oceny. Uważaj na eksplozję kombinatoryczną!

`CrossValidator` — k-fold CV w Spark

```
from pyspark.ml.tuning import CrossValidator
from pyspark.ml.evaluation import BinaryClassificationEvaluator

evaluator = BinaryClassificationEvaluator(
    labelCol="is_fraud", rawPredictionCol="rawPrediction", metricName="areaUnderPR"
)

cv = CrossValidator(
    estimator=pipeline,          # 👈 cały Pipeline, nie tylko model!
    estimatorParamMaps=grid,
    evaluator=evaluator,
    numFolds=3,
    parallelism=2,               # 👈 ile kombinacji trenować równolegle
    seed=42,
)

cv_model = cv.fit(train_df)
best_model = cv_model.bestModel   # to już PipelineModel!
```

> 💰 **Koszt:** `numFolds × len(grid)` pełnych treningów Pipeline'u. 3 foldy × 9 kombinacji = **27 treningów**!

⚡ `TrainValidationSplit` — szybsza alternatywa

```
from pyspark.ml.tuning import TrainValidationSplit

tvs = TrainValidationSplit(
    estimator=pipeline,
    estimatorParamMaps=grid,
    evaluator=evaluator,
    trainRatio=0.8,                # 👈 pojedynczy split zamiast k-fold
    parallelism=2,
    seed=42,
)

tvs_model = tvs.fit(train_df)
```

	`CrossValidator`	`TrainValidationSplit`
Liczba treningów	`numFolds × len(grid)`	`len(grid)`
Solidność oceny	Wysoka (k-fold)	Niższa (jeden split)
Kiedy używać	Mniejsze dane, mniej kombinacji	Bardzo duże dane, szybka iteracja

🕵️ Class Imbalance — problem, który psuje model po cichu

Fraudy to **1-2%** transakcji. Model chętnie "poświęci" tę małą klasę, żeby zminimalizować ogólny błąd!

```
df.groupBy("is_fraud").count().show()
# +--------+------+
# |is_fraud| count|
# +--------+------+
# |       0|493200|
# |       1|  6800|
# +--------+------+
```

> 🔑 **Zasada kciuka:** Gdy klasa pozytywna < 10% danych — standardowe metryki i domyślne wagi klas Cię oszukają!

⚖️ Radzenie sobie z imbalance: `weightCol`

Zamiast fizycznie usuwać dane (undersampling), możemy **ważyć przykłady** podczas treningu:

```
from pyspark.sql import functions as F

n_fraud = train_df.filter("is_fraud = 1").count()
n_ok = train_df.filter("is_fraud = 0").count()
fraud_weight = n_ok / n_fraud   # 👈 im rzadsza klasa, tym większa waga

train_weighted = train_df.withColumn(
    "class_weight",
    F.when(F.col("is_fraud") == 1, fraud_weight).otherwise(1.0)
)

rf = RandomForestClassifier(
    featuresCol="features", labelCol="is_fraud", weightCol="class_weight"
)
```

> 💡 Model "widzi" każdy przykład fraudu jakby wystąpił wielokrotnie — bez fizycznego duplikowania danych!

Undersampling — alternatywa w Spark

```
majority = train_df.filter("is_fraud = 0")
minority = train_df.filter("is_fraud = 1")

fraction = minority.count() / majority.count()
majority_sampled = majority.sample(fraction=fraction, seed=42)

train_balanced = majority_sampled.union(minority)
train_balanced.groupBy("is_fraud").count().show()
```

Podejście	Zalety	Wady
`weightCol`	Zachowuje wszystkie dane	Nie każdy model wspiera wagi
Undersampling	Prosty, szybszy trening	Tracimy dane większości klasy

🌲 `GBTClassifier` — Gradient-Boosted Trees

Kolejne drzewa **poprawiają błędy** poprzednich — często lepsze wyniki niż `RandomForestClassifier`, ale wolniejszy trening sekwencyjny.

```
from pyspark.ml.classification import GBTClassifier

gbt = GBTClassifier(
    featuresCol="features", labelCol="is_fraud",
    maxIter=50, maxDepth=5, seed=42,
)

gbt_model = gbt.fit(train_df)
predictions = gbt_model.transform(test_df)
```

Model	Trenowanie drzew	Zwykle najlepsze przy
`RandomForestClassifier`	Równolegle, niezależnie	Duże, szumowe dane
`GBTClassifier`	Sekwencyjnie, poprawiając błędy	Mniejsze różnice klas, precyzja

> ⚠️ `GBTClassifier` w MLlib obsługuje **tylko klasyfikację binarną** — idealny do `is_fraud` (0/1)!

📊 AUC-ROC vs AUC-PR — którą wybrać?

```
roc_eval = BinaryClassificationEvaluator(labelCol="is_fraud", metricName="areaUnderROC")
pr_eval  = BinaryClassificationEvaluator(labelCol="is_fraud", metricName="areaUnderPR")

print(f"AUC-ROC: {roc_eval.evaluate(predictions):.4f}")  # np. 0.94 — świetnie! 🎉
print(f"AUC-PR:  {pr_eval.evaluate(predictions):.4f}")   # np. 0.42 — hmm... 🤔
```

Metryka	Uwzględnia	Wrażliwa na imbalance?
AUC-ROC	TPR i FPR (w tym True Negatives)	❌ Mało — TN dominują i "ukrywają" błędy
AUC-PR	Precision i Recall klasy pozytywnej	✅ Bardzo — pokazuje realną jakość na rzadkiej klasie

> 🔑 **Przy fraud rate 1-2%, zawsze patrz najpierw na AUC-PR!**

💾 Zapis i odczyt `PipelineModel`

```
best_model = cv_model.bestModel

# Zapis
best_model.write().overwrite().save("/tmp/cardshield_best_model")

# Odczyt (np. w innym procesie / na produkcji)
from pyspark.ml import PipelineModel
loaded_model = PipelineModel.load("/tmp/cardshield_best_model")

predictions = loaded_model.transform(nowe_transakcje)
```

> 💡 `PipelineModel` zapisuje **wszystkie** stage'y (indexery, assemblery, model) — nie musisz nic odtwarzać ręcznie przy wdrożeniu!

🚀 Skalowanie CV: cache i partycje

```
train_df = train_df.repartition(8).cache()   # 👈 CrossValidator odczytuje train wielokrotnie!

cv_model = cv.fit(train_df)
```

Czynnik	Wpływ na czas CV
`numFolds`	Liniowy — więcej foldów = więcej treningów
Rozmiar siatki (`len(grid)`)	Liniowy — każda kombinacja to osobny trening
`parallelism`	Więcej równoległości = szybciej, ale więcej RAM na klastrze
Brak `.cache()` na danych treningowych	Spark przelicza dane od nowa przy **każdym** foldzie!
Zbyt mało/dużo partycji	Złe rozłożenie pracy na executory

> ⚠️ Brak cache'owania danych treningowych to najczęstsza przyczyna "dlaczego mój CrossValidator trwa wieczność"!

⚠️ Najczęstsze pułapki

Pułapka	Dlaczego to problem	Rozwiązanie
Ocena tylko przez AUC-ROC na niezbalansowanych danych	Metryka "kłamie" — wygląda świetnie mimo słabego modelu	Zawsze sprawdzaj też AUC-PR
Zbyt duża siatka `ParamGridBuilder`	Eksplozja liczby treningów, godziny czekania	Zacznij od małej siatki, rozszerzaj stopniowo
Brak `.cache()` przed `CrossValidator`	Wielokrotne, kosztowne przeliczanie danych	`train_df.cache()` przed `cv.fit()`
Próg decyzyjny 0.5 "z automatu"	Model rzadko przekracza 0.5 dla rzadkiej klasy	Dostosuj próg do kosztów FP/FN biznesowo
Ważenie klas bez przemyślenia wag	Zbyt duża waga = model "przewrażliwiony" na fraud	Zacznij od inverse frequency, testuj warianty

Key Takeaways

1. ✅ `ParamGridBuilder` + `CrossValidator` automatyzują tuning hiperparametrów
2. ✅ Przy class imbalance zawsze patrz na **AUC-PR**, nie tylko AUC-ROC
3. ✅ `weightCol` to elegancki sposób radzenia sobie z niezbalansowanymi danymi
4. ✅ `GBTClassifier` często bije `RandomForestClassifier`, ale trenuje się wolniej
5. ✅ Cache'uj dane treningowe przed `CrossValidator` — inaczej zapłacisz za to czasem

🔮 Co dalej?

Na dzisiejszych **ćwiczeniach** zbudujesz pełny pipeline z tuningiem hiperparametrów do wykrywania fraudów kartą płatniczą na ~500 000 transakcji! 💳🕵️

Wykład 11 (15.12.2026): Cloud Computing i GCP — BigQuery, Dataproc ☁️

❓ Pytania?

Dziękuję za uwagę! 🎉

Zajęcia nr 10 | 08.12.2026